

Date 15 / Aug / 2025

Intro to Java: Hello World

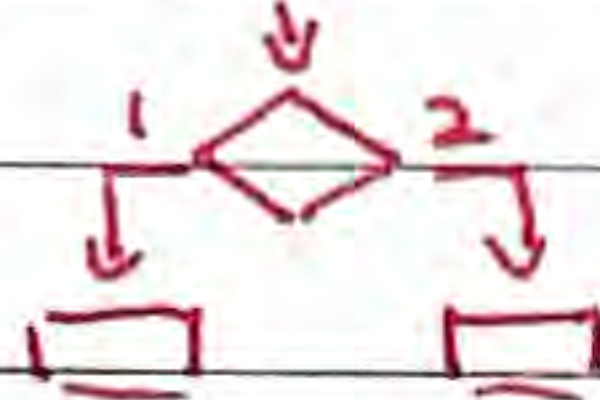
Algorithms - Step-by-step instructions that accomplish a task
(1.1)

Consists of a combination of the following:

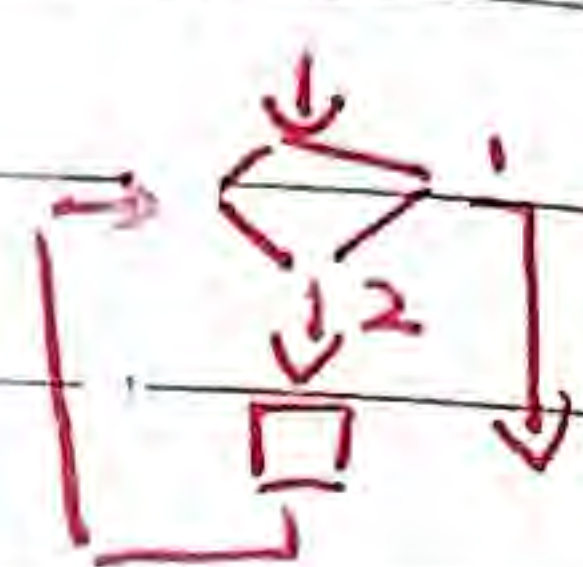
1. Sequencing



2. Selection



3. Iteration



May be written in code, pseudocode, or flowcharts

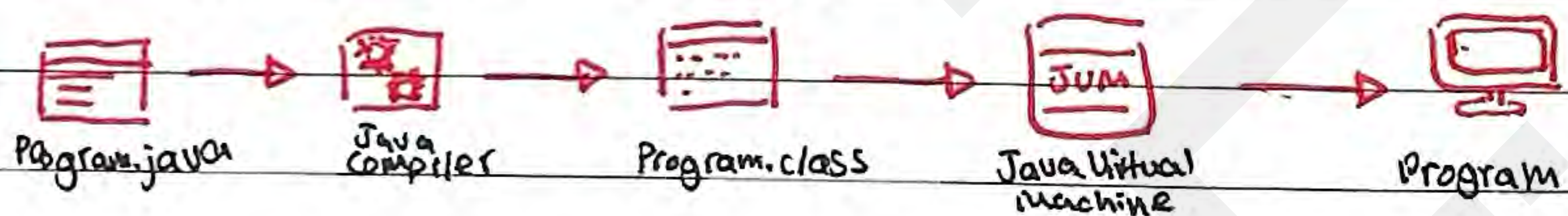
Integrated Development Environment (IDE)

> App installed on your computer, typically providing the following:

1. Text Editor 2. Compiler 3. Execution Environment 4. Debugging Tools
(runtime)

Compiler - Special program that translates code written in one programming language into machine language

- Checks code for errors before it is run



- Some languages are compiled, some are interpreted
Java, C++ Python, ruby

The most simple Java program:

```

public class HelloWorld ← class
{
    public static void main (String[] args) ← main method
    {
        System.out.println ("Hello, World!"); ← print
    }
}

```

String literal - collection of characters within double quotes

println - Prints literally the string (no changes)

Eg. code after return statement

3 types of errors

1. Syntax / Compiler Error

Eg. system.out.print ("Hi")

2. Exception ← mid-program (cannot perform)

Eg. division by 0 (runtime error)

3. Logic error ← programming error

Eg. typo → stops during runtime

System.out.println() - Does a line break after print

i.e. HelloWorld
HelloWorld

System.out.print() - Does not do a line break after print

i.e. HelloWorldHelloWorld

Syntax - Part of the language in which the compiler checks

Style - Convention widely adopted by software developers, for readability purposes

Date 19 / Aug / 2025

Variable - A named container that stores a specific type of value boolean, not bool

- Primitive types include int, char, double, boolean, etc.
- Object types include String.

others include
char, byte,
short, long,
float

↑
class, not
primitive type

Variable names should describe what is stored in them

A variable must be declared before use, with its type and name

The assignment operator "=" assigns a value to the variable

int stores integers

short stores short integers

char stores characters

long stores long integers

byte stores bytes

double stores decimals

String stores strings

float stores

objects reference
another container storing
actual data

String works like any other object, but it can...

- Use "" to store literal string constants
- Use + or += to concatenate strings

Camel Case is a style used

- First word lowercase, every word after is written in titlecase

• E.g. thisIsAVar

Escape character

- Makes the character after a backslash function in a special way

`\n` = newline

`\\` = backslash "\"

`\t` = tab

`\"` = quotation mark ""

Arithmetic

`+` = add

`-` = subtract

`/` = divide

`*` = multiply

`%` = modulo

Precedence of operators is same as in maths

(modulo in same rank as "*" and "/") ^{← same rank} performed left to right

- You can use parentheses to force a specific order

The type of the result is determined by the types of the operands. E.g. $5 / 2 \Rightarrow 2$; $5.0 / 2 \Rightarrow 2.5$

- When one operand is a double, other is int, result is double

Math class (used for more mathematical operations)

Math.abs(x) $|x|$

Math.max(x, y)

Math.round(x) round x

Math.min(x, y)

Math.pow(x, e) x^e

Math.random()

Math.sqrt(x) $\sqrt{x}$

etc. (trig)

null means empty, can be stored into objects

E.g. String a = null;

Unix-based Command Line

Most operating systems, like Linux, MacOS, Android, are unix-based systems.

The terminal plays a powerful role

some commands include...

(shortcuts : enter = do command
tab = complete command)

> pwd displays working directory

> ls argument lists items in argument (if argument is empty, then working directory)

> cd argument change directory to argument

(if argument is empty, then change directory to one before)

(if argument is /, then change directory to root)

↓
start with / = absolute path

Scanner - Used for console input

- To use this, you need to import the package first

```
import java.util.Scanner;
```

← before classes

To use the scanner, you must also declare it

```
Scanner sc = new Scanner(System.in);
```

var name from where?

* Then, you can retrieve the input as ...

var names

```
int sc.nextInt();
double sc.nextDouble();
String sc.nextLine();
boolean sc.nextBoolean();
```


Date 27/Aug/2025

Variable types

When doing operations with `int`, the result will also be an `int` (will be rounded if stored in an `int`)

`double a = 3 / 2;` $\Rightarrow$ `a` is `1.0` (not `1.5`!)

Casting - Changing the type of a value

- For hardcoded values, you can just change it to a double by adding a `.0`

- But for variables?

$\Rightarrow$ Use casting operators

() has precedence over arithmetic operations
wanted type

$\rightarrow$ or `*1.0`

E.g. `double a = (double) 3 / 2;`

3.0
1.5

When casted as an `int`, a double will just be truncated (rounded down)

E.g. `(int) 3.9  $\Rightarrow$  3`

Note: Storing a double into an `int` without casting creates an error

"possible loss of conversion"

$\hookrightarrow$ Compiler error: will not compile

Variable types

You can also take advantage of (int) acting as a floor to perform rounding.

`int rounded = (int) (double_value + 0.5);`

* E.g. $3.2 + 0.5 = 3.7$ (rounds down when truncated) ^{→ 3}

$3.7 + 0.5 = 4.2$ (rounds up when truncated) ^{→ 4}

Range of variable types

ints and everything digital are finite!

ints have the range from -2^{31} to $2^{31} - 1$ ^{~2B}

going beyond this range is called overflow

- Overflow is not detected by the Java compiler or interpreter

- Overflow is a logical error

Java compiler = (compiles and runs without exception)

integers (binary)

1 1 1 1 1 1 0 1 = -3
^{sign bit (1 if -, 0 if +)}

Integers are stored in a byte

✓ makes arithmetic easier

Numbers of different signs but same magnitude have bits that are flipped (1 to 0, 0 to 1)

Compound assignment operator

You can simplify syntax when updating a Variable (i.e. using variable, doing operation, store it back)

$a = a + b;$ is $a += b;$

$a = a - b;$ is $a -= b;$

$a = a * b;$ is $a *= b;$

$a = a / b;$ is $a /= b;$

$a = a \% b;$ is $a \% = b;$

For increments of one, you can also use...

^{or}
decrease

$a++$

($a += 1$)

$a--$

($a -= 1$)

Random number

$\text{Math.random}()$ returns a random double over $0 \leq \text{Math.random()} < 1$

So, to return an integer over $[\text{min}, \text{max}]$ you can use:

$> (\text{int}) (\text{Math.random()} * (\text{max} - \text{min} + 1)) + \text{min};$

APIs and Libraries

Libraries - Collection of prewritten code programmers can use to simplify development processes, avoid "prebuilt functions" rewriting common functionality, and leverage tested, optimized implementations of algorithms

Packages - Mechanism for organizing related classes into a logical namespace. Software libraries are called by different names across various programming languages, e.g. modules, packages, extensions.

- Java collects packages in .jar files, to easily import them into projects. Java ARchive
- when using these classes, you can call attributes (data) or behaviors (methods)

Application Programming Interface (API) - Specific type of documentation that describes how a programmer can use the classes in a library

(in modern computing, can also refer to 3rd party product/service to enable external developers or other apps to access functionality and data)

Abstraction - Process of hiding complex implementation "simplification" details while exposing only the essential information needed to use a piece of code

- When used in a method, procedural abstraction
- Libraries and APIs facilitate abstraction

Comments - Notes written in non-machine language inserted in the source code

Used to: Document program details

Describe parts of code (e.g. methods)

Explain obscure or unusual places in the code

Temporarily exclude fragments of code for debugging

Types of comments: "//" single line comment

"/*" block / multi-line comment
*/"

Javadoc Comments

Used by JDK's special utility program javadoc to auto generate documentation in HTML format from source code

Should precede class, method, or field

Can use special tags: @param (parameter of method)
@return (return value of method)

Also used at the very start of a .java file,

@author, @version, etc.

Special kind of block comment, start with

/**, end with */, and * at the beginning of each line in

the comment

/**

*

Comment here

*

@param Parameters

*/

Preconditions - Conditions that must be true before execution of a section of program code in order for method to behave as expected (preconditions might not be checked for within the method)

- Typically written in comments of an FRA

Methods

A method can be declared with the following format:

public [static] returnType methodName (type0 name0, ...)

↓
visibility
(who can call the method)

- public = any class, anywhere
- private = only inside same class
- protected = same package and sub classes

↓
return type,
e.g. int, String, double, etc.
void returns nothing

↓
"name" of the method (identifier), help identify method and what it does

↓
parameter list, type and name of inputs, can be empty

Static or (ownership)
(non-static)

- static means one shared value or function (global one)
- non-static means a separate value or function per every object (requires new object and similar to "for this clone only")

A
run B
output
①

A
new B
output
①

B
output
②

Calling methods

Static methods:

methodName() if in the same class

class Name. methodName() if not in same class

Calling Methods

non static: requires creating an object (similar to clone in scratch)

- When you want the object to run a method, do

object Name. methodName()

(no matter if in same class or not because object Name was created with a specified class)

Arguments - Values passed to parameter variables when a method is called

↳ refers to just the local variables inside a method

- Number, order and types of arguments must match the method's parameters
- Calling a method with no parameter still requires "()"

Overloaded Methods - Methods in the same class can have the same name but then they must differ in number or types of parameters.

- Compiler automatically figures out which one to call
- Example of polymorphism, same piece of code behaving differently depending on actual type of object

- E.g. String.substring()

can be begin to end, or just begin

(i.e. string[a:b] or string[a:])

Any code after a return statement will cause a compiler error. The program won't compile at all.

Attributes - Data stored in variables defined in a class (e.g. position)

Behaviors - What instances of the class can do (or what can be done with them), defined by methods.

Method Signature - Method name and parameter list

- Used in documentation
- E.g. `void println(String x)` (documentation for println where input is a string)

Call by value initializes the parameters with copies of the arguments

- When calling a method, the method retrieves the values of the arguments as copies, does not use original (so any change in the copied variable does not affect original)

String class

- There are some common methods used in the string class

`substring(beginIndex, endIndex)` $\Rightarrow$ Get substring in a string
 variable name begin index (inclusive) end index (exclusive)
 $\rightarrow$ similar to a `[begin: end]` in other languages

`substring(beginIndex)` $\Rightarrow$ Get substring in a string until very end
 variable name begin index (inclusive)
 $\rightarrow$ similar to a `[begin:]`

Class methods

- A static field (class field or class variable) is a single field shared by the entire class and all objects of the class
- A non-static field belongs to an individual object of a class

similar to "clones" in scratch

← typically global constants

- Public static fields are referred to in other classes using "dot notation": `ClassName.constName`

E.g. `Math.PI`, `Color.BLUE`, `System.out`

Static methods

- Can access and manipulate the static fields of a class
- Static methods cannot access non-static fields or call non-static methods of the class
- Called using "dot notation": `ClassMethod.staticMethod(...)`

Math class is all static (no need to create objects)

Object-oriented programming

> models the application as a world of interacting objects

- Objects can represent real-world objects (e.g. product), software entities (e.g. events), abstract concepts (e.g. rules of a game)

Class - A piece of the program's source code that describes a particular type of object

↑
"blueprints"
to create
objects

Classes and Objects

An object is an instance of a class

- A program can create more than one object(s) of the same class, typically using "new"
- Instantiation - Act of creating an object

E.g. String name = "Bob";

↳ This specific type does not require "new"

↳ Requires class (type), reference, object
"string" "name" "Bob"

Variables and Values (all memory is stored in R.A.M.)

- When a variable is declared, Java creates a place in memory to store a value

- When we assign a value to a primitive type, the value is stored in the variable.

- When we assign a value to a variable that holds an object, the object is created in available memory and a reference to that location is stored in the variable.

i.e. `object = new Class(<data>)`

↙ e.g. with strings
each variable only
stores a reference to
the memory of string

Assignment operator

- Stores a value in a variable, "="
- Works same way with primitive types and with objects

E.g. Fraction f1 = new Fraction(3, 7);

Fraction $f_2 = f_1$

only one Fraction object (since only one "new")

↳ However, there are two references to that one Fraction object.

data:

—fi—

Fraction@23e09cd1 → Fraction:

f2: Fraction@23e09cd1 $\rightarrow$ $n=3$
d=7

Objects

initializing object without definition
 e.g. `Class a;` is same as
 → initializing with null (e.g. `Class a=null`)

You can initialize an object with

`ClassName nameVar = new ClassName (arguments)`

To do so, you must already have a library with
 class `ClassName`

```
public class ClassName
```

```
{
```

<code>

```
}
```

the code typically involves
 attributes, constructors (optional)
 and methods

if no special requirement,
 you can leave out constructors

attributes (instance variables)

constructors (to initialize attributes if necessary)

methods (returns, actions, behaviors of instance)

Just like methods, you can overload constructors

(e.g. have one as default without params and another
 of the same name with params)

E.g. `public class Student`

```
{
```

```
private String name, email;
```

← student attributes

```
public Student ()
```

```
{ <code>
```

```
}
```

```
public Student (String initName, String initEmail)
```

```
{ <code>
```

```
}
```

← student constructors

```
public String getName ()
```

```
{ return name; }
```

← student methods

```
public void Print ()
```

```
{ <code>
```

```
}
```

```
}
```


Objects

When you print an object directly, it will print the reference id of the object

E.g. `Circle c = new Circle();`

`System.out.print(c);`

output: `Circle@6ceda1dc`

↑ implicitly calls toString

What's also interesting is, usually, when you print an empty variable it results in an error.

But when you call an empty variable attribute of an object in another class, no error occurs and prints "default" values (int = 0, double = 0.0, bool = false, String = null)

E.g. `public class Circle`

`{`
`public double r;`
`public boolean a;`
`public int n;`
`public String x;`

main:

`Circle c = new Circle();`
`System.out.println(c.r);`
`System.out.println(c.a);`
`System.out.println(c.n);`
`System.out.println(c.x);`

Default Values

↑ or any other object

output:

0.0
false
0
null

default values (only when initialized outside method)

`String .toString()`, converts an object into a textual string

- This might be annoying since `c.toString()` calls the conversion method in String class
 - You can overwrite this behavior by adding a custom `.toString()` method into the wanted class (e.g. `c`)
- ⇒ When java expects a string, `toString` is called automatically

Anatomy of a class

```

import ... — import statements
public class MyClass — Class header
{
    Fields — attributes/instance variables
    Constructors — procedures for instantiating a new object
    Methods — actions/behaviors of the object
}
    
```

Declaration of a field:

private [static] [final] datatype name;

↑ field is class variable or not
 ↑ constant?
 ↑ double, int, etc.
 ↑ non-static means unique to each instance

Declaration of a constructor:

- looks like a method, but no return value
- name of constructor must be same as that of a class
 - ✓ you can have 0 parameters in constructors too
- ↳ You can have multiple constructors but each must at least vary in the number/type of parameters

Java provides a default, no-args constructor

ONLY if no constructor is present

⇒ Class Name varName = new Class Name (<args>);

• We call constructors with the "new" operator

↳ if the constructor is improperly called, it will result in a compiler error

if a constructor with params are present then you can no longer call the constructor without args unless another constructor without params was declared

Accessor method

- Used to return private values/attributes in a method

↳ Useful because prevents "outside code" from editing the attribute whilst allows for reading

↳ when a method returns an object, a copy of the reference is returned, not of the object

Mutator method

- Used to edit or update private values/attributes in a method

↳ Useful because prevents "outside code" from editing the attribute however they like

> We as the programmer can decide how attributes change

- An example of how we achieve encapsulation

↳ we hide the details inside

Method declaration/signature:

public [static] returnType methodName(type0 name1, ...)

↑
ownership
(runs method
in all or one
instance)

↑
double,
int, etc.

↑
Identifier
name

↑
parameter list
(can be empty)

Calling methods

- instance methods are called on specific objects using the dot operator

e.g. a.doMethod();

In java, you can call a method that returns a value and NOT store that value

↳ no error (except maybe logic/programming error)

↳ However, putting ";" or a variable "a;" will result in a compiler error.

String Class

- Belongs to java.lang
- Stores a string of characters
- You can concatenate strings, unique to the string class
- Like other objects, strings also have constructors

Strings are objects

object variable $\rightarrow$ string object $\rightarrow$ string class $\rightarrow$ object class

reference

reference

reference

e.g. toString()



String class inherits from Object class (all classes inherit unless otherwise stated)

Literal strings - Anonymous constant objects of the String class $\Rightarrow$ defined with ""

↳ Can be passed to methods and constructors as parameters

↳ Have methods you can call e.g. cmd.equals("start")

Recall "escape" characters that begin with a backslash

new line, \t tab, \n backslash

or System.out.printf(); to print

With String.format(" ", ...)

e.g. %.2f

you can easily format a string (%s = String, %d = int, %f = %.f, %b = boolean, etc.)

of decimal's rounded

↳ between "" you type string normally, but substitute in the % and type for variables and put the rest of the parameters as the variables separated by commas

String class

Strings are immutable

- Methods return a new string, not changing the original
- Immutable objects are useful because several references point to one object, avoids causing issues with changing object itself

E.g. `String s1 = "sun";`
`String s2 = s1;`

`s1` } → "sun"
`s2` }

If you want to change a string, you have to assign a new string to the variable

`s = x` → "sun" → 
 ↳ "Sun"

- Disadvantage: less efficient
 - You need to create a new string and throw away the old one for every small change
 - If you need to modify it often, use a StringBuffer

Empty string - Has no characters, length of 0

- Not the same as uninitialized string, which is null

String constructors

- string constructors are not used much, since literal strings and assignment operators do the same thing

No args: `String s1 = new String();` or `String s1 = "";`

Copy constructor: `String s2 = new String(s1);` or `String s2 = "" + s1;`
 ↳ creates another object instead of reference

String methods

• String indices start at 0

> s.length() variable name length of a string (# of characters)

overloaded { > s.substring(a, b) substring from a (inclusive) to b (exclusive)
same as `s[a:b]` in other languages

> s.substring(a) substring from a (inclusive) to end
same as `s[a:]` in other lan

* Note, the max. argument is `s.length()` or one above the max. index for both a and b

`s.substring(s.length())` returns "" would be runtime error, out of bounds

`s.substring(0, s.length())` returns entire string

> s.concat(s1) concatenates s1 to s (or `s += s1`)

Same as `s + s1` where s1 does not have to be another string, an int or double will be automatically converted to String and concatenated

overloaded { > s.indexOf(a) returns index of a character (the first it appears in) or a substring

or

> s.indexOf(a, b) returns index of the character / substring after index b (inclusive)

if "character a or substring a isn't present (or not found after index b), it returns -1

String methods

Comparisons

`s1.equals(s2)`

↳ same string?

`s1.equalsIgnoreCase(s2)`

ignores upper case

`s1.compareTo(s2)`

↳ s1 is before s2?
(alphabetically)

`s1.compareToIgnoreCase(s2)`

Cases

`s.toUpperCase()`

`s.toLowerCase()`

Concatenation and addition go from left to right

↳ Addition will occur between int/double and int/double

↳ Concat will occur between String and any other value

E.g. `s = 3 + 1 + "a"`, is `s = "4a"`

`s = "a" + 3 + 1`, is `s = "a31"`

* Because strings are immutable `s.method()` does not do anything to the string and must be reassigned

E.g. `s = s.method()`

`.equals()` is not the same as `'=='`

`.equals()` compares actual string

`'=='` compares references (what's stored in variables)

Programming flow charts

Elements of a flow chart



start/end
point



arrows
(shows relationship)



input/output



process (method / code statement)



decision (selection)

should always have
one true, one false
arrow stemming out

Boolean primitive data type

↳ boolean variables may have only two values: true or false

↳ You can define boolean fields or boolean local variables

E.g. `boolean hasMiddleName = false;`

You can use if and else-if statements
as a form of selection

↳ `if (<boolean expression>)`

{

<statements>

}

↑
for boolean variables
there's no need to put true/false
(since they already are boolean)!

OR

↳ `if (<boolean expression>)`

{ <statements> }

else

{ <statements> }

← you can use else if as well

Boolean Expressions

- Evaluates to either true or false
- Written using boolean expressions, relational operators, logical operators

> Six Relational Operators

< less than > greater than

<= less than or equal to

>= greater than or equal to

= equal to

!= not equal to

* You can compare primitive types together
('a' == 65)

> You can apply relational operands to numbers or chars. Using == or != with doubles may result in rounding errors.

E.g. double x = 7.0;

double y = 3.5;

if (x / y == 2.0) {
}

may be false

the solution is to round beforehand

Relational Operators and objects

Relational operators do not compare objects, they instead compare the reference variables

E.g.

if (cmd == "help") { } is never true

↑
whatever is stored here is not stored in the same location as "help"

• equals is inherited from Object
↑
all objects have them unless overridden

* You should use String.equals() to compare strings (values)
if (text != null)

* Use == or != only to compare same object or null

if and if-else statement

```
if (<condition>)
{
    <statements>
}
```

only executes when condition is met

two-way conditions

```
if (<condition>)
{
    <statements>
}
else
{
    <other statements>
}
```

one or the other will happen (but only one can happen)

You can also have else if ...

```
if (<condition>) {<statement1>}
else if (<condition>) {<statement2>}
else if (<condition>) {<statement3>}
... ..
```

only one of these statements will be executed

else {<statement n>} * optional else

When you only have one single statement per selection, the curly brackets are not necessary

```
if (<condition>)
    do something;
```

must use { and } when multiple statements

Nested conditionals/selection

Multiple if statements, inside one another conditional statements.

```
if ( ) {
    if ( )
    else
else { }
```


Logical Operators

& & "and"

↑ same as `bool1 == bool2`
similar to arithmetic operators,
these are binary operators
(two operands)

- Both conditions must be true for logical operation to return true.

↓ || "or"

- Either one (or both) condition(s) can be true for logical operation to return true

! "not"

↑ unary operator
(only takes one operand)

- The logical operation returns the exact opposite boolean value

E.g. `(boolA & & boolB)` is true when both boolA and boolB are true

`(boolA || boolB)` is true when either boolA, boolB, or both are true

`!boolA` is true when boolA is false

Rank of operators

Highest

! - (unary) ++ -- (cast)

* / %

+ -

< <= > >= == !=

& &

Lowest

||

AND takes precedence over OR

Ternary Operator

`<condition> ? <valueIfTrue> : <valueIfFalse>`

DeMorgan's Laws

Distributive property for logical statements

$\text{not } (A \text{ or } B)$ gives $\text{not } A$ not or $\text{not } B$
or... $\text{not } A$ and $\text{not } B$

similarly

$\text{not } (A \text{ and } B)$ is equivalent to $\text{not } A$ or $\text{not } B$

You're "negating" a relational operand

You can verify DeMorgan's laws with truth tables

E.g. $\text{!(p \& \& q)}$ = (!p || !q)

	P = true	false	
q = true	false	true	
q = false	true	true	

	p = true	false	
q = true	false	true	
q = false	true	true	

Short circuit evaluation

• If the first condition guarantees a result, the second won't be evaluated

i.e. $\text{if } (\text{condition1} \&\& \text{condition2})$

if condition1 is false, then condition2 won't be evaluated

if $(\text{condition1} || \text{condition2})$

if condition1 is true, then condition2 won't be evaluated

this means that you can have condition2 be division by 0 or other error-causing statements as long as condition1 guarantees return already

↳ $\text{if } (a == \text{null} || a.\text{indexOf}("") == -1), \text{if } (x != 0 \&\& 5/x == 1)$

↳ avoids
NullPointerException

↳ avoids Arithmetic Exception

Overriding .equals Method

- As discussed earlier, comparison between two objects must utilize .equals () to avoid just comparing the reference.
- Classes inherit this from the Object superclass and, to override, it must follow the following declaration

```
public boolean equals (Object other)
```

```
{
```

```
    return other != null &&
```

```
        (other instanceof ClassName) &&
```

```
        ...
```

```
}
```

↑ if we don't use "Object",
we will overload

* Why use "Object"? The method from object superclass uses Object type params. This makes our method override it. If we use our specific class instead (e.g. ClassName) it will overload and only compare objects of the same class with our method. E.g.

```
ClassName a = new ...;  
String b = "...";  
System.out.print(a.equals(b));
```

↑ uses default
method instead

↳ We typically want to return false if the two objects are of different classes so using "Object" allows us to compare that... we use instanceof as a logical

operator

a instanceof ClassName

is 'a' an object of 'ClassName'?

Computational thinking practices

1. Designing Code - "What should the program do when..."

2. Developing Code - Code can react, repeat, or change with conditional statements

3. Analyzing Code - Develop test cases that cover every possible pathway in our code

4. Documenting Code and Systems - Conditionals and loops can become hard to understand. Document with clear and helpful comments.

5. Using Computing Responsibly - The decisions we make in our program can affect real people

Loops

Iteration - Repetition of the same code fragment several times

Java has three control statements: for, while, do-while

while loop

← logical/boolean expression

while (condition) ← while (True) is forever loop

does this
when condition
is true →

<statements> ← body of the loop

} ← if only one statement, braces optional

while loop involves 3 phases:

Initialization (values tested in the condition must be initialized with value)

Tests first, → Testing (while loops check conditions before each iteration)
So if condition not true then Change (at least one of the variables should be changed to
none of code inside is run avoid infinite loop)

Useful if iteration until certain condition is met.

for loop

• shorthand combining initialization, condition, and change

for (initialization; condition; change)

{
<statements>

e.g.

int i = 1; i <= 100; i++

← after every iteration, we do this

The variable you initialize inside is local for the loop (outside is undefined)
• if you initialize it outside and write k=0, then it'll be defined

temporary variable we initialize before loop

for what condition do we iterate?
If false, we break

for loop good for iteration of known amount of times

Loopsdo-while loop

Same as while loop except you iterate before you check

- Guaranteed at least one iteration

```
do
{
```

<statements>

if condition not met,
next iteration not executed

```
} while (condition);
```

Useful when variables tested in the condition are calculated or read in the body of the loop

Also used when user interactions (interact until valid)

break, return, continue

break - instructs program to immediately quit the current loop and go to first statement following the loop

return - instructs program to immediately quit the current method and return to the calling method

continue - instructs program to immediately quit the current iteration and go to the next (if condition valid)

Loops

Recursive Functions/Methods

You can define a method that calls itself to also achieve iteration.

E.g. `public static long factorial(int n)`

`{ return (n == 1) ? 1 : n * factorial(n); }`

Same as `a if ( ) else b` in python

Examples for loops

`while (n > 0)`
do {
} while (n > 0)

`for (int i = 0; i <= n; i++)`

Advanced Algorithms

Algorithm - A more or less compact, general, and abstract step-by-step recipe that describes how to perform a certain task or solve a certain problem

• Represented by flow charts, series of steps

• Properties:

• Compact - Can use iterations or recursions to repeat steps

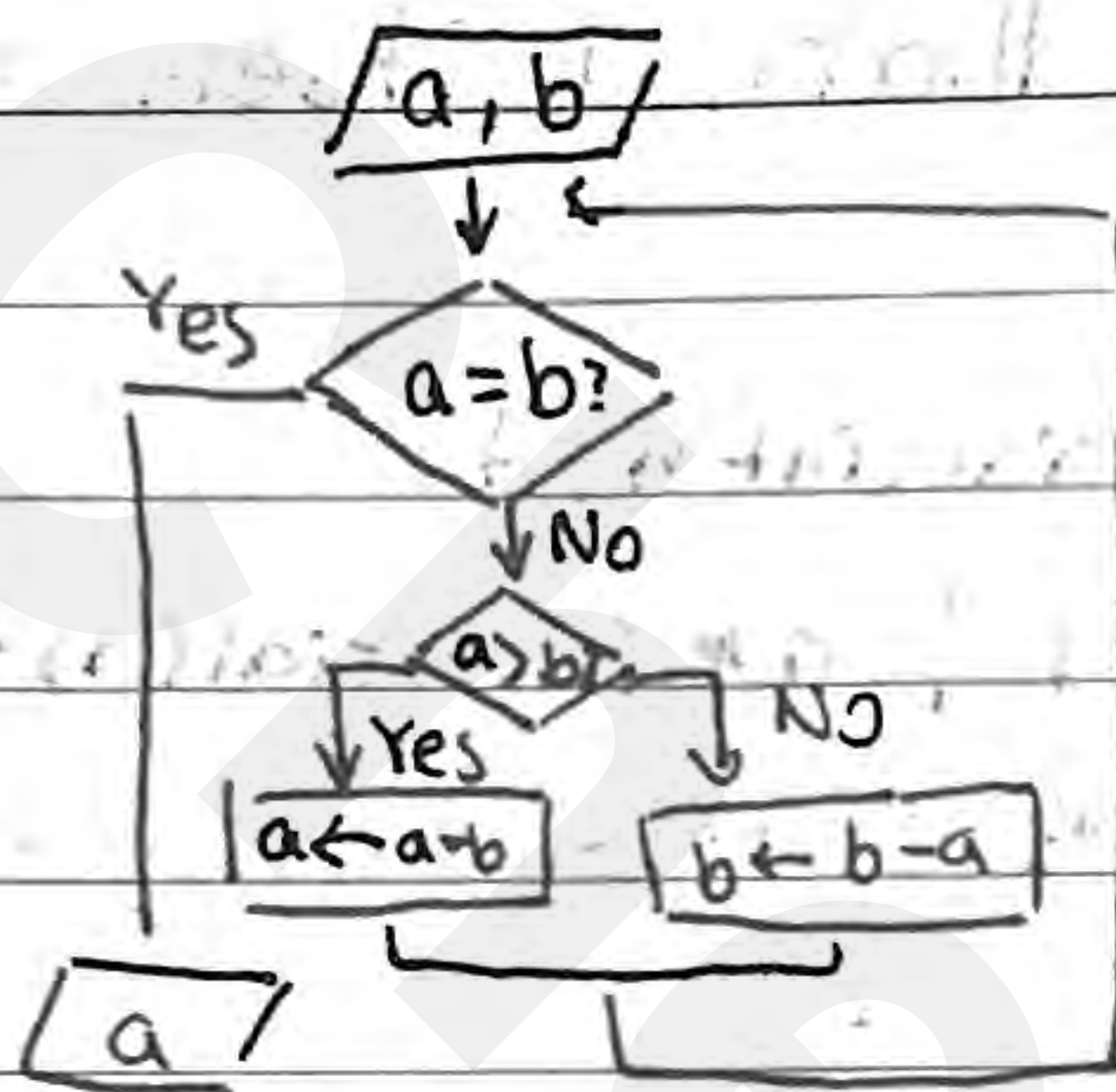
• General - Same algorithm applies to any "size" of task/input

• Abstract - does not depend on a particular computer language or platform

Advanced Algorithms

Euclid's Algorithm

- Finding greatest common factor of two positive integers



```

public static int gcd(int a, int b)
{
    while (a != b)
    {
        if (a > b)
        {
            a -= b;
        }
        else {
            b -= a;
        }
    }
    return a;
}
  
```

Nested Iteration

- Loops found in other loops
- Composed of outer and inner loops
- Typically used in matrix structures (higher dimensional grids)
- Brace brackets are optional when the body of the loop(s) is one statement (if a single for loop in another, that still counts as one statement)
- "break" breaks out of current loop (so if in innermost then breaks out of inner loop)
- Triangular nested loops, when you start inner loop at the outer loop's loop control variable

The temp. variable in for loops

Program Design

Object-Oriented program models application as an interaction between objects

- Useful for...
 - code reusability and modularity
 - modelling real-world systems
 - Code maintainability
 - Collaboration

- disadvantageous for...
 - performance overhead
 - steep learning curve
 - program size and complexity
 - unsuitable for some problem types
 - coupling issues (interconnected classes)

Encapsulation - Scope of data should be confined to the object which it is defined (as much as possible)

↳ Using private variables → limits side effects and dependencies

↳ Protect internal state of an object

Abstraction - Hiding complex implementation details and exposing only the essential information

↳ when done in a method, it's classified as procedural abstraction

Abstraction can be achieved with encapsulation

↳ what is exposed

↳ how can it be protected

Data Abstraction

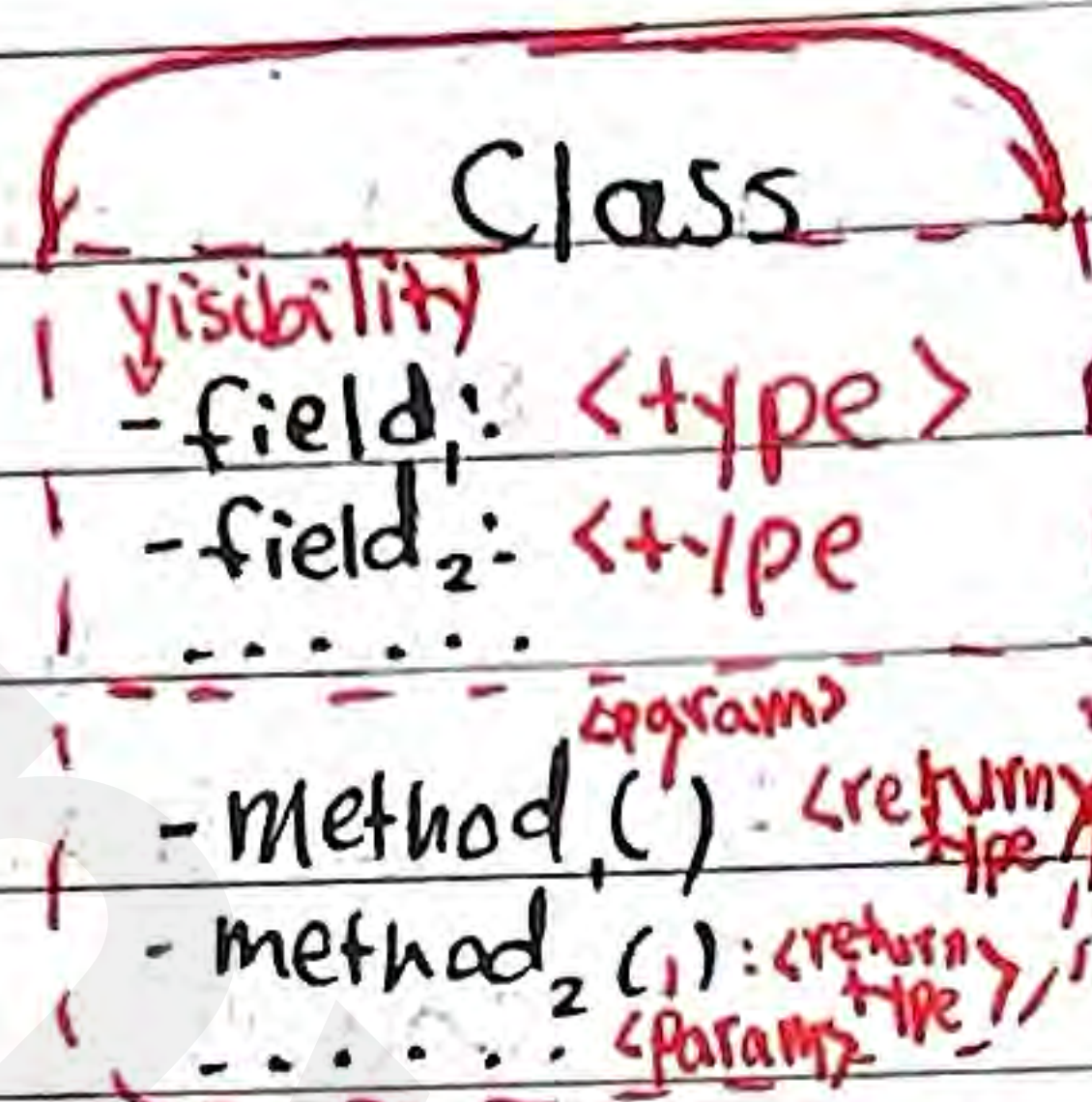
Lists are examples of data abstraction

→ We use it without understanding how computer understands it.

UML Class Diagram Resources

class diagram format

- Visibility "-" private
- "+" public
- "#" protected
- "~" default



association can be any class

inheritance & association (inheritance specific for subclasses)

Animal in italics denote

Class relationships

Inheritance (subclass)

child → parent

Tortoise → Animal → cat

inherits properties from animal (and when animal is changed, so are methods/attributes passed down)

• Dependence (calls methods from another class)

• Aggregation (can be part of another class but not necessarily)

one —◇ number

• Composition (child cannot exist without parent)

class1 —◆ class2

• Multiplicity (how many instances of one class per other class)

0..1 0 to 1; 1; 1..* one to many

eg-

customer — 1 — 0..* order

At least 0 orders per customer, = 1 customer per order

Encapsulation and Information Hiding

- A class interacts with other classes only through constructors and public methods
- other classes do not need to know mechanics (implementation details) of a class to use it effectively
- Encapsulation facilitates teamwork and program maintenance (making changes to code)

Java Arguments

- Arguments are passed by value for primitive types

```
↳ public static void inc(int a) {
    a++;
}
```

running `x = 3`
`System.out.print(inc(x));`
 prints 4, but `x` is still 3
 (Value is copied!!)

Modifying parameter variable does not affect original variable's value

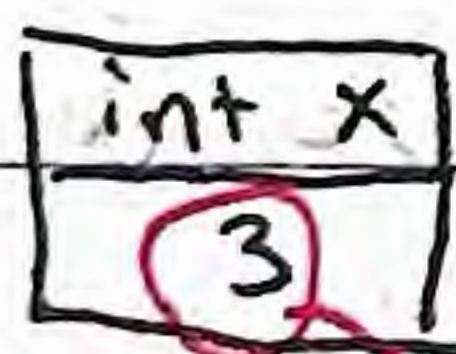
- Arguments are passed by reference for objects

```
↳ public static void inc(A a) {
    a.val++;
}
```

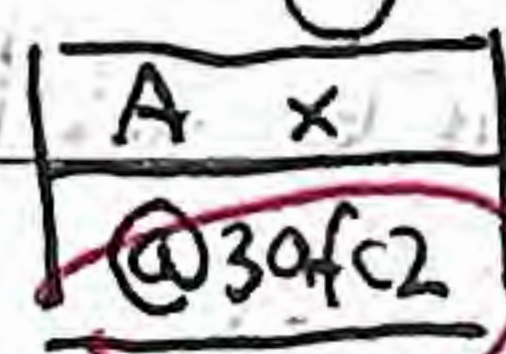
running `A x = new A(3)`
`System.out.print(inc(x));`
 prints 4, and `x` is 4 too
 (reference is copied)

Hence, modifying the object parameter modifies actual original object

In other words, whatever is stored in the variable is copied and passed as argument



and



Passed to argument

Static Fields

• A static field (class field, class variable) is a single field shared by all objects of the class

↳ In other words, static fields belong to the class, not object

↳ can be used to hold a constant used by all objects of the class
↳ or statistics or totals for all objects of the class

• A non-static field (instance field, instance variable) are object-specific

↳ In other words, non-static fields belong to the individual object

Public static fields (e.g. Global Constants) can be referred to through "dot notation" in other classes:

ClassName.constName

E.g. Math.PI, Color.BLUE ← typically all-caps convention though no requirement

↳ These are final variables though public static variables and methods will also work with dot notation

Static fields are usually not initialized in constructors (but can be)

↳ Typically, they're initialized in declarations or public static methods

If a class only has static fields, there's no point in creating objects of that class since all of them would be identical

↳ That's why Math, System have no public constructors and hence cannot be instantiated

Static Methods

- Can access and manipulate static fields of a class
 - ↳ But cannot access non-static fields or call non-static methods of the class
 - ↳ Called using "dot notation", `ClassName.staticMethod()`

Hence...

Non-static Methods
Constructors

> can access both static and nonstatic fields

Static methods > can access only static fields

Scope

- Each variable has a scope, the area in the source code where it can be used
 - ↳ A use outside its scope results in syntax error
 - ↳ As long as scopes don't overlap, two variables can have same name

this includes
↑ different objects of same class!

Fields or instance variables - whole class

↳ Declared outside all constructors and methods

↳ local variables do not have access modifiers (i.e. 'public', etc.)

Local variables - From its declaration down to the closing brace of the block in which it is declared

Eg.

```
if (true) {
    int x = 5;
}
```

 ↳ includes declaration within methods, constructors, for loops, ifs, etc. — anything within "{}" except the largest class "{}"

↳ Once constructor, method or block is exited, local variables lose their values and are destroyed

using if without {} does not allow for any variable to be initialized inside

Scope

↳ The scope thus for any variable is within the "{ }" it is defined in

```
public class Test {
    private int x;
    ...
}
```

}

whole class

```
public class Test {
    public void method() {
        int x;
    }
}
```

}

only method

"this" keyword

"this" is reference to current object or the object whose method or constructor is being called

↳ Improves readability of code by referencing the calling object inside its class

↳ It can be used to pass the calling object as an argument to a method

E.g.

```
private int x;
public void set(int x) {
    this.x = x;
}
```

Note:

If "this" isn't used and there is a confusing statement, the variable refers to the local variable, not instance variable

or...

```
public class A {
    public void set() {
        B.do(this);
    }
}
```

```
public class B {
    public static void do(A a) {
        ...
    }
}
```

i.e.

```
public void set(int x) {
    x = x;
}
```

Not instance variable x

Scope

Some concepts to note:

• Private variables and methods can be accessed as long as it is in the same class. This includes other objects in the same class

• You can also have parameters that are objects of the same class

E.g.

```
public class Example {  
    public void method (Example other) {  
        ...  
    }  
}
```

Arrays

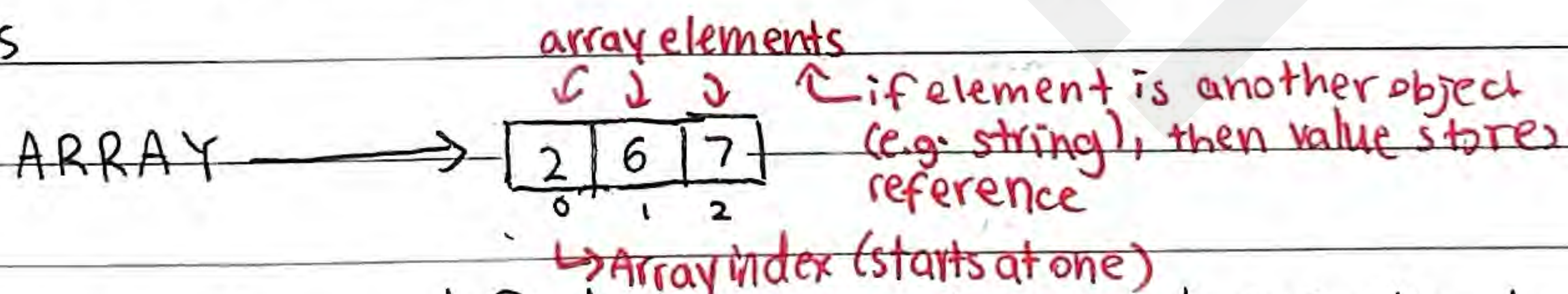
A constant block of consecutive memory locations that hold values of the same data type

→ We can find any element in the array given its index without having to traverse all of it, making it super fast

→ The consecutive memory storage nature of the array makes changing the length of it impossible

In Java, arrays are objects with no methods

↳ Their variable stores a reference to the array, like all objects



↳ Hence arrays can be modified when passed through a method (since reference is passed)

Arrays

Declaring arrays require a type and name

↳ To denote arrays, add "[]" to the end of the element type

↳ An array can only have elements of the same type

↳ Arrays can be set as a literal list or an empty one with "new" and length

- Empty ones set with "new" contain default values of the specified type (e.g. 0.0 for double, null for String)

E.g. `String[] x = new String[n];`

- Creates an empty array (null) of n Strings
 ↳ "`{ }`" used instead of "`[ ]`"

`int[] x = {1, 3, 5, 7, 9};`

- Creates a literal list array of set int values

You can also create arrays of other objects

E.g. `Circle[] circles = new Circle(10);`

where circle is custom object

Accessing a specific element requires array and index

↳ Indices start at zero and accessors are written as follows:

>

array Name [index]

e.g. `x`, `names` ←

↳ `0, 1, 2, ..., index.length - 1`

Arrays

Elements of an array can be directly modified and treated as a variable

E.g. `arrayName[i] = someValue`

sets arrayName's i-th element to someValue

↳ Though, like all variables, ensure the value types match

The length of an array is given by `length`

`arrayName.length` ⇒ int value

* Note it lacks "()" compared to string lengths. This is because length is a public instance final field

Array Index OutOf Bounds Exception

↳ Runtime error when index is invalid when trying to refer to an array element

↳ This includes index -1, `arrayName.length`,
remember array starts at zero so only valid (exclusive) up to length

Like any other variable, arrays must be initialized before use

E.g. `Example[] arrayName;`

`arrayName = ...;`

`...`

`arrayName[i] ...;`

Arrays

As any other object, arrays are passed to a method as a reference.

The elements of the original array are not copied and are accessible in the method's code.

↳ Hence, no new arrays are created in memory as an array is passed to a method.

Arrays can be returned from a method.

↳ Return type follows from the type we use when initializing them, i.e. adding "[]" to the end of element types.

↳ E.g. `String [ ]`, `boolean [ ]`

Traversing an array

Accessing each element in the array using iteration.

↳ With a for loop, you can match loop variable with the array indices.

E.g. `for (int i = 0; i < arrayName.length; i++)`
array indices
↓ start at zero
↑ non inclusive
of length index

↳ A while loop is also possible though lacks the convenience of having a readily-available loop variable as in the for loop.

Arrays

Array traversal can also use a "foreach" loop

↳ Rather than iterating over indices, it iterates over actual values (or references stored in each element)

↳ Works for arrays and lists

to assign the elements
↓
to a new variable

```
for (<type> variableName : arrayName) {
```

// calling the current element the loop is traversing
// only requires calling "variableName"

```
}
```

↳ Remember, each element is copied by value/reference to the loop variable

Hence, you cannot change the values of an array element directly (whether primitive or references)

↳ It will only modify the local loop variable

↳ "For each" also doesn't allow you to access the index of an element, nor does it allow you to add or remove elements (for lists)

↳ However, since objects are passed as references

• Mutable objects can be modified through public mutator methods

* An array can be of mixed objects if you use "Object[]" as type

Text Files

Files can be retrieved and read by a program using the File and Scanner classes

↳ The Scanner class is in the java.util package
i.e. java.util.Scanner or java.util.*

↳ The File class is in the java.io package
i.e. java.io.File or java.io.*
• where "io" stands for input/output

A file is retrieved by creating a File object

↳ Constructor takes in: File(<String pathToFile>)

• path can be relative or absolute ← The file

E.g. File myFile = new File("data.txt");
constructor does not check for if file exists

In turn, a Scanner object can be created to read the file

↳ Constructor takes in a File object: Scanner(<File fileObj>)

↳ If the file cannot be found, the scanner constructor will throw a FileNotFoundException
(type of IOException)

Java will not let you compile if you leave a Scanner constructor reading any file (since file might not exist)

↳ Even if it exists, Java still won't let you compile

↳ If you only create a File object, program will compile still.

It's only if you try to Scanner the File object.

FileNotFoundException

To allow the program to compile, you must account for the potential exception.

One way to do this is using "throws exceptionType" statement to the header of the method that uses the Scanner on the file.

E.g.

```
public static void main (String[] args) throws FileNotFoundException
{
    File myFile = new File("data.txt");
    Scanner scan = new Scanner(myFile);
    ...
}
```

Using "throws" statement only indicates that the method may throw an exception, allowing the program to compile.

↳ However, when exception is thrown, program will still terminate.

FileNotFoundException is a subclass of IOException

↳ Using "throws IOException" will also handle "FileNotFoundException"

↳ But to use either exception, you must need to import these exceptions:

```
java.io. FileNotFoundException
```

```
java.io. IOException
```


try &

<code>

↳ Code is attempted to be executed

3

Catch (<exceptionType variable>){

<code>

↳ i.e. "FileNotFoundException e"

3

"IOException e"

(optional) (finally { codes })

"Exception e"

Code guaranteed
to run after try-catch
(even if catch block has
errors itself)

* type of exception is stored in the variable "e" as an exception object

↳ i.e. code after try-catch might still skip if some errors are not handled within try-catch. Finally block guarantees code within are run.

Reading in Data with Scanner

<scanner object>. nextLine()

- Returns next line of text up until the end of the line ("n") as a String (consumes newline but only returns string before, excluding, "n")

`(Scanner object).next()`

- Returns the next line of text up until a whitespace (" ")

- You can set a custom delimiter with: `<scanner object>.useDelimiter(↑);`
`<String pattern>`

Reading in Data with Scanner

<scanner object>.nextInt()

- Returns the next int read from input source
- If next int does not exist, then InputMismatchException
- It does not read until the end of the line, so calling nextLine() is needed to consume newline character (but nextLine() itself will return anything before newline, so likely empty in most cases without the newline character itself)

<scanner object>.nextDouble()

- Returns the next double read from input source
- If next double does not exist, then InputMismatchException

<scanner object>.nextBoolean()

- Returns the next boolean read from the input source
- If next boolean does not exist, then InputMismatchException

<scanner object>.hasNext()

- Returns true if there is a next item to read in the file or input source; false otherwise
- Useful when used in while loops to scan entire file or input source

<scanner object>.close()

- Closes the input stream and releases system resources previously used by <scanner object>

Reading in Data with Scanner

Each line may have several types of data (e.g. each row has several columns in a table)

↳ In .csv, each line is separated by commas

We can interpret the different data in a single line by separating them with .split():

↳ "," for csv

`<String line>.split(<delimiters>);`

which returns an array of Strings with each element representing a field of data that was separated by the delimiter.

E.g.

`String[] words = sentence.split(" ");`

↳ In turn, we can convert the string elements to integer or double with `Integer.parseInt(<string>)` or `Double.parseDouble(<string>)`

Wrapper Classes

- A class that wraps (encloses) around a primitive data type and gives it an object appearance
- It's functionality is essentially the same as a primitive data type, except parameters and variables are passed by reference, not value

This is useful to pass to methods that expect an object, or put in a collection like an ArrayList which can only hold objects as elements

For integers and doubles:

Integer *<name>* = new Integer(*<value>*);

Double *<name>* = new Double(*<value>*);

↳ for int, passing doubles won't work

↳ for doubles, passing int also work

Like strings, directly setting the object to a value also works

Integer *<name>* = *<value>*; Double *<name>* = *<value>*;

↳ cross-type won't work here

↳ *<value>* must be double, cannot be integer

Integers and Doubles function

mostly the same as int and double

due to automatic conversion by Java

Wrapper classes

Autoboxing - Automatic conversion the Java compiler makes between primitive types and their corresponding object wrapper classes

↳ E.g. Integer i = 2;

autoconverted to "new Integer(2)"

Unboxing - Automatic conversion the Java compiler makes from the wrapper class to the primitive type.

↳ E.g. int i = new Integer(2);

autoconverted to "2"

They are unboxed then autoboxed

This also allows for

<Integer Object> ++

<Integer Object> += <value>

*Same with

<Double Object> instead

<Integer Object> <, <=, etc. <Integer Object>

However, they behave differently because of their object nature

• When a method takes in Integer or Double object, they are passed by reference, not value

↳ Passing an int or double makes compiler autobox which can make it seem passed by value

Wrapper Classes

Equality operators won't work with wrapper classes

- Unboxing won't happen, so "==" and "!=" will compare by reference, not value
- Use .equals() on wrapper objects instead

However, wrapper classes are immutable

- When an object is incremented/changed with an arithmetic operator, it creates a new object

e.g. Integer a = 1; a → 1
 Integer b = a; b → 1
 b++; b → 1+1 (new object) while a → 1
 ~~b → 1~~

Casting also won't work...

E.g. (int) (new Double(<value>)) is a compile error

Nor does cross-type assignment work...

E.g. Double d = 5 is a compile error

You can, however, manually unbox a wrapper class with instance methods

- <Integer Object>.intValue()
returns type int
- <Double Object>.doubleValue()
returns type double

Wrapper Classes

Integer and Double classes also have static fields and methods.

Integer.MAX_VALUE - returns maximum 32 bit integer
(overflow)

- Any greater integer (e.g. `Integer.MAX_VALUE + 1`) gets rolled over starting from the minimum value (i.e. `Integer.MIN_VALUE`)

Integer.MIN_VALUE - returns minimum 32 bit integer

- Any smaller integer (e.g. `Integer.MIN_VALUE - 1`) gets rolled back from the maximum value (i.e. `Integer.MAX_VALUE`)
(underflow)

You can also convert a string to number

`Integer.parseInt(<string>)` → Throws `NumberFormatException`
- Returns string conversion to int when string is not int

`Integer.valueOf(<string>)`
- Returns string conversion to Integer object
* `Integer.valueOf(<int>)` is a manual way to box int values, converting them to Integer object

Wrapper Classes

Double.parseDouble(<string>)

- Returns string conversion to double
(string can be of both decimal or integer)

Double.valueOf(<string>)

- Returns string conversion to Double object
- * Double.valueOf(<double>) is a manual way to box double values, converting them to Double object

ArrayList

- A re-sizable list which uses an underlying array and creates new ones if capacity exceeded

↳ By default, array size 10 is created

↳ Once internal array is full, ArrayList automatically resizes by creating a new array 1.5x the old one, then transferring and disposing the old array

Size - How many elements are actually stored in an ArrayList

Capacity - The length of an ArrayList's underlying array

ArrayList

Requires import statement

```
> ! import java.util.ArrayList;
```

To construct one, you declare it like an object

```
ArrayList < <type> > <name> = new ArrayList< <type> > ();
```

↑
e.g. Integer, String, or any other object

- Declaring an ArrayList requires object type
- You can set the initial capacity different to the default 10:

```
ArrayList< <type> > <name> = new ArrayList< <type> > (<capacity>);
```

E.g. `ArrayList<String> a = new ArrayList<String>(25);`

- You can also create an ArrayList of mixed types with "ArrayList<Object>"

ArrayList

Common Methods

> `<ArrayList>.size()`

↳ Returns int number of elements in the list

> `<ArrayList>.add(<element>)`

↳ Adds `<element>` to the end of the list

↳ Also returns true

> `<ArrayList>.remove(<int index>)`

↳ Removes the item at the index and shifts higher index items to the left (decreased index)

↳ Also returns the removed element

E.g. `a.remove(2)`

before: 1 2 ³ 3 4 5 6
after: 1 2 4 5 6

> `<ArrayList>.add(<int index>, <element>)`

↳ Overloaded method, moves any current objects at `<index>` or beyond to the right (to a higher index) and inserts `<element>` at `<index>`

E.g. `a.add(2, 7)`

before: 1 2 3 4 5 6
after: 1 2 7 3 4 5 6

ArrayList

Common Methods

> `<ArrayList>.get(<int index>)`

• Returns the item in the list at `<index>`

> `<ArrayList>.set(<int index>, <new element>)`

• Replaces the original item at `<index>` with `<new element>`

• Also returns the "old" element that was previously at the specified position

You can also use enhanced and regular for loops to iterate over an ArrayList

Regular For Loop:

```
for (int i = 0; i < <ArrayList>.size(); i++) {
```

```
    ...
```

```
    ... <ArrayList>.get(i);
```

```
}
```

*Note, if you add elements while looping, the for loop might not traverse all elements

Note that
size() auto-
matically
adjusts each
iteration

• If you remove elements, you might skip elements or cause Index Out Of Bounds Exception if you remove the last elements and call it after words on a single iteration.

ArrayList

Enhanced For Loop:

```
for (<element type> <name> : <ArrayList>) {  
    ...  
}
```

* Note that if you structurally modify the ArrayList (i.e. add or remove elements) during iteration, the program will throw "ConcurrentModificationException"
↳ If you need to modify it, use a regular For or while loop

ArrayList indices start at 0 and end at $\text{<ArrayList>.size()-1}$

• You use $\text{<ArrayList>.get(<int index>)}$ instead of "[]"

Similar to an array, if you try to call an index outside of the valid range, the code will throw:

Index Out Of Bounds Exception
(not ArrayIndex Out Of Bounds Exception)

2D Arrays

Useful when data is organized by rows and columns

- Similar to 1D arrays but include two dimensions

- E.g. `int [][]`

Hence, retrieving a specific element requires two indices:

E.g. `arrayEg [2] [3];`

gets element at row 3, column 4

↳ You can also use it to set a specific element

`arrayEg [2] [3] = <value>;`

Declaring a two dimensional array

Empty array:

`<typeName> [][] <var> = new <typeName> [<w>] [<l>];`

- Declares an empty 2D array of type `<typeName>` of size `<w> x <l>`

Initialized array:

`<typeName> [][] <var> = { { i1, i2, i3, ... }, { ... }, ... };`

arrays within an array

2D Arrays

- > Row and column length
 - Using `<array>.length` directly returns the number of rows of 2D array `<array>`
 - ↳ The number of "arrays" in the 2D array
 - To get the number of columns in a rectangular array, we call `.length` on an array within the 2D array `<array>` instead:
 - E.g. `<array>[0].length`

Array algorithms

- Calling a 2D array with one dimension returns an array (a row)
 - E.g. `int[][] arr = { {1, 2, 3}, {4, 5, 6} };`
 - `arr[0] ⇒ {1, 2, 3}` and `arr[1] ⇒ {4, 5, 6}`
- This is why `arr[0].length` returns the number of columns in a rectangular array
 - ↳ And thus `arr.length` returns the number of rows

2D Arrays

Loops and traversal

You can use a nested regular for loop

```
<type> [][] <name> = { { ... }, ... };
```

```
for (int i = 0; i < <name>.length; i++) { ← By rows
```

```
    for (int j = 0; j < <name>[0].length; j++) {
```

```
        <name>[i][j]; ↑ Each element in a row
```

```
    }
```

```
}
```

This is a Row-Major traversal

(traversals occur across each row)

You can invert this into a Column-Major traversal

(occurring down each column) by using

"<name>[j][i];" instead.

Enhanced for-loops similarly also work

```
<type> [][] <name> = { { ... }, ... };
```

```
for (<type>[] <row> : <name>) { ← each array in 2D array
```

```
    for (<type> <element> : <row>) {
```

```
        <element>; ↑ each element in the inner array
```

```
    }
```

```
}
```

Enhanced for-loops naturally follow Row-Major traversal, and is very difficult to convert into Column-Major instead

Theoretically, inner and outer loops can be mixed for and enhanced for

*For enhanced for-loops, the same limitation where you can't modify the array value directly still applies

Searching and sorting

Sequential Search

- Scans the list item by item, comparing target value to each element

```
public int sequentialSearch(Object[] arr, Object target)
{
    for (int i=0; i < arr.length; i++) {
        if (target.equals(arr[i])) {
            return i;
        }
    }
    return -1;
}
```

- The maximum number of comparisons is n , for $n = \text{arr.length}$
 - ↳ To establish target value not in array
- This is $O(n)$ (order of n) algorithm efficiency

Searching and Sorting

Binary Search

- The elements of the list must be arranged in ascending (or descending) order
- Target value is always compared with the middle element of the remaining search range
- We must have random access to the elements of the list

Iterative Implementation

```
public int binarySearch(int [] arr, int value) {
    int left = 0;
    int right = arr.length - 1;
    while (left <= right) {
        int middle = (left + right) / 2;
        if (value == arr[middle])
            return middle;
```

You can use `compareTo` for Strings

```
    else if (value < arr[middle])
```

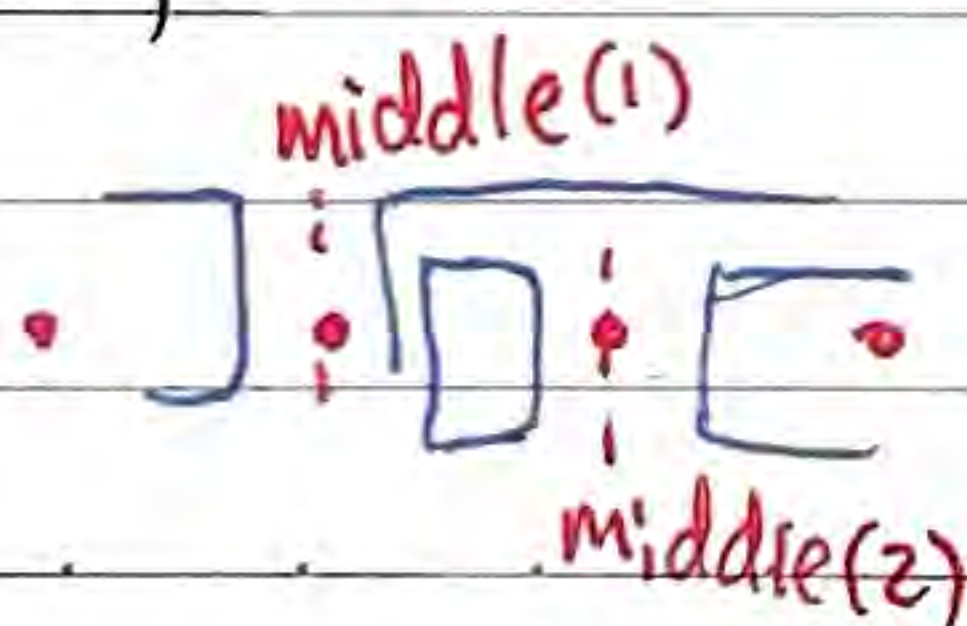
```
        right = middle - 1;
    else
```

```
        left = middle + 1;
```

```
    }
```

```
    return -1;
```

```
}
```



← indices of leftmost and rightmost elements

← average of left, right (and takes floor)

← takes left half by setting right bound the one before middle (just checked)

← takes right half by setting left bound the one after middle (just checked)

Searching and Sorting

Binary Search

Recursive Implementation

```
public int binarySearch (int[] arr, int value, int left, int right)
{
    if (right < left)
        return -1;
    int middle = (left + right) / 2;
    if (value == arr[middle])
        return middle;
    else if (value < arr[middle])
        return binarySearch (arr, value, left, middle - 1);
    else
        return binarySearch (arr, value, middle + 1, right);
}
```

Has an $O(\log_2 n)$ efficiency

- Maximum number of iterations required is $\lceil \log_2 n \rceil$

compareTo

useful to declare a class as comparable because it is called from library methods (E.g. `Array.binarySearch(object[] arr)`)

compareTo is an abstract method defined in the `java.util.Comparable<T>` interface

↳ To implement it for an object, "import `java.util.Comparable`" and declare class as "public class `<Class>` implements `Comparable<Class>`"

: Method declaration follows "public int compareTo (`<Class>` other)"

`obj1.compareTo(obj2)`

positive if `obj1 > obj2` 0 if equal
negative if `obj1 < obj2`

Sorting

- Rearranging elements of a list in ascending or descending order

Selection Sort

- Iterate through entire list and find max/min

1 13 8 5 2 1 3
 ^{max}
 n

- Swap the element with the first/last element

1 3 8 5 2 1 13

- Repeat again but now ignoring the element(s)

1 3 8 5 2 1 13
 ⁿ⁻¹

you just moved to the front/end

Selection sort iterates through all elements no matter what. Hence efficiency is always the same for n elements (even if already organized)
 $\rightarrow O(n^2)$ algorithm

Insertion Sort

- Average number of comparisons is roughly half of the number in Selection Sort

- The best case is when the array is already sorted: takes only $(n-1)$ comparisons

- Worst case is $n(n-1)/2$ comparisons (array is sorted in reverse order)

- Generally an $O(n^2)$ algorithm (but faster than selection sort)

Sorting

Insertion Sort

- Expand sorting range one-by-one
- For every each new element you look at, insert it in the correct space for the range before it

i.e. 1, 3, 5, 2, 13, 1, 8

k=1

k=2

k=3

k=4

* 2 is out of place, insert before 3

1, 2, 3, 5, 13, 1, 8

etc.

Merge Sort

- Repeatedly splits array in half until single elements
- Then single elements are compared in pairs, merged, then compared again with other elements until final array is constructed

38, 27, 43, 3, 9, 82, 10

split

38, 27, 43

3, 9, 82, 10

split

split

3, 9, 82, 10

merge $\Rightarrow$ 27, 38, 43

merge $\Rightarrow$ 3, 9 $\Rightarrow$ 3, 9, 10 $\Rightarrow$ 3, 9, 10, 82

• Takes roughly $n \cdot \log_2 n$ comparisons

• Best case when array already sorted ($n-1$ comparisons)

3, 9

3, 9, 10

3, 9, 10, 27 $\Rightarrow$... 3, 9, 10, 27, 38, 43, 82

• $O(n \log_2 n)$ algorithm

Recursion

- A recursive method calls itself, must contain at least one base case that halts recursion
- Each recursive call contains its own set of local variables and parameters
 - ↳ Parameter values capture the progress of a recursive process, like loop variables
- Any recursive solution can be written iteratively
- Any iterative solution can be written recursively